Learned KV-Cache Eviction Forgets Facts in Multi-Turn Dialogue

Technical report, work in progress (draft of 28 October 2026)

Nijat Mammadov Eötvös Loránd University (ELTE), Budapest nijat.mmmadov@gmail.com

October 2026

Abstract

Learned KV-cache eviction (KVP) has been evaluated with the cache compressed once after the context is read; in a chat the cache is cut back after every turn, and we ask whether facts from early turns survive. On a synthetic multi-turn needle test with Qwen2.5-0.5B-Instruct and 256 retained tokens per KV head, KVP agents we trained with Apple's code and fine-tuned on recall data answer 4 of 112 word facts and 5 of 120 digit facts, counted on the dialogues the full cache answers (112/120 and 120/120); the H2O heuristic answers 59/112 and 15/120. Forcing parts of the fact to stay in the cache shows that the answer tokens are not enough: adding the short phrase linking the fact to its subject raises word recall from 35 to 88 of 112 under KVP, and from 0 to 116 of 117 under StreamingLLM at 1.5B. Keeping the whole user turn in 24 random heads of 48 gives 40/112; in all 48, 112/112. Restoring the lexically retrieved user turn into all heads from a CPU archive, with the same retained GPU budget, recovers every fact on both sets. Equal-size controls, selected head subsets and retrieval under paraphrase remain open. Code, trained agents and per-dialogue results will be released at `https://github.com/nijatmammadov/kvp-multiturn-recall`.

1 Introduction

Eviction methods keep the KV cache of a language model at a fixed size by deciding which cached tokens to drop. KVP [1] learns this decision: a small reinforcement-learning agent per KV head ranks cached tokens by predicted future attention from keys, values and positions alone, and it beats heuristic baselines on RULER and on OASST2 perplexity, with the cache compressed once after the input is read. In a chat, eviction repeats after every turn: a fact stated early must survive every later cut, and the question that needs it arrives only at the end. SCBench [2] evaluates KV-cache methods on multi-turn shared-context workloads; KVP postdates it. We test KVP under repeated conversational eviction and find:

- Our KVP agents recall 4/112 word and 5/120 digit facts; H2O, 59/112 and 15/120 (§3.1).
- Answer tokens are not enough; adding the 3–4-token linking phrase lifts KVP from 35 to 88 of 112 (§3.2).
- Recall grows with heads holding the whole user turn: 40/112 at 24 of 48, 112/112 at 48 (§3.3).
- Restoring the retrieved turn into all heads recovers 112/112 and 120/120 at the same GPU budget (§3.4).
- Answer-token survival alone is not sufficient for recall (§3.5).

2 Setup

Models. Qwen2.5-0.5B-Instruct (48 KV heads) and Qwen2.5-1.5B-Instruct (56 KV heads) [3]. Apple's KVP weights are not released, so we trained the 48 agents for 0.5B with Apple's code (commit `d33c2dc`) and the base model frozen: 4,000 steps each on 400 OASST2 [4] training traces (of 500; 50 validation, 50 held out), then a 1,500-step warm start on those 400 plus 500 synthetic recall training traces (of 600; 50/50 validation/held out), keeping the checkpoint with the best validation reward. The reward scores how well an agent ranks tokens by future attention in recorded traces, not answer correctness under repeated eviction. On held-out OASST2 traces the agents rank tokens better than StreamingLLM on 41 of 48 heads, but better than an accumulated-attention (H2O-style) ranking on only 28 of 48 (mean cost 1.876 vs 1.895). Apple trained each agent for the same 4,000 steps but on about 4,500 OASST2 samples [1]; how our agents compare with theirs is unverified. An OASST2-only control for the extra fine-tuning was not completed (compute ran out), so any change after fine-tuning cannot be attributed to the recall data rather than to extra training. *All KVP results below use these fine-tuned agents*; in an earlier bf16 run the agents before fine-tuning did no better on words (0/119 vs 2/119 at $k = 4, 8$). There are no KVP agents for 1.5B, so tests at that size use StreamingLLM as the base.

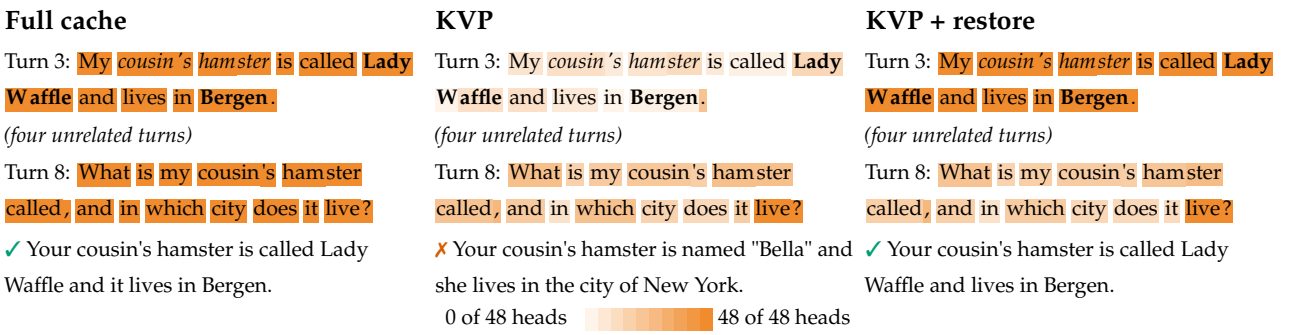


Figure 1: Tokens still in the cache when the question arrives, for one word-set dialogue (view after [1], Fig. 1). Shade: share of the 48 KV heads holding the token. Bold: answer tokens; italic: linking phrase; below each panel, the model's actual answer.

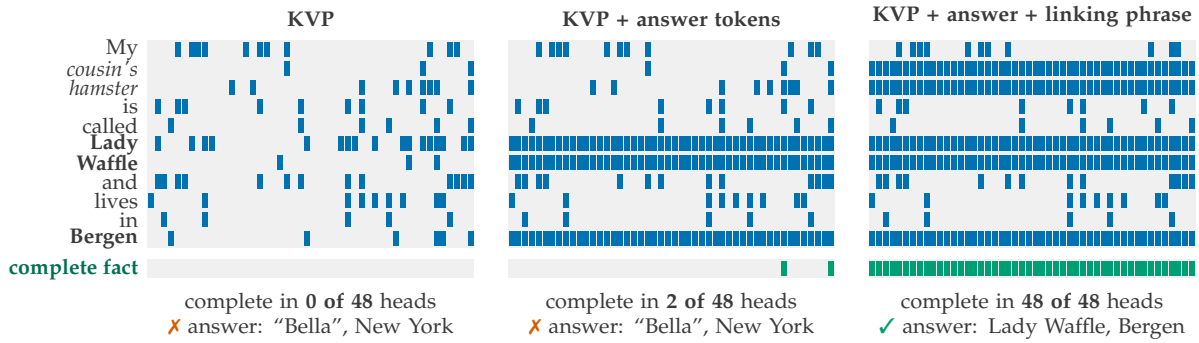


Figure 2: Where does each of the 48 KV heads keep the fact when the question arrives? Same dialogue as Fig. 1. Rows: words of the fact sentence (bold: answer; italic: linking phrase); columns: the 48 heads, layer 0 on the left. Blue: the head still holds the word. Green: the head holds the complete fact, linking phrase and answer together. In the second and third panels those words are force-kept in all heads. Layout after [1], Fig. 11.

Eviction. Evicted keys and values are deleted. After each user message and each reply, every KV head is cut to 256 tokens; the first 4 and last 16 are always kept. Between cuts the cache grows by the new turn. Baselines: full cache, StreamingLLM [5] (first and most recent tokens) and H2O [6] (highest accumulated attention, with the same first/last protection). All methods get the same user messages but write their own replies. Greedy decoding, fp32, one NVIDIA T4.

Task. Each synthetic dialogue has two unrelated OASST2 user turns, the fact as turn 3, $k \in \{4, 8\}$ more unrelated turns, then the question; the model replies to every turn (about 1,000 tokens per dialogue). *Word* facts name an invented pet and its city (correct if the answer contains both and no other listed city); *digit* facts give a five-digit code (correct if the extracted code matches). Each set has 120 dialogues. To locate what must survive, we force the answer tokens, the answer plus the linking phrase ("cousin's hamster"), or the whole user turn (including chat-template tokens) to stay in all heads or in n random heads.

Statistics. Experiments ran in batches with decision rules fixed before each batch; later hypotheses followed earlier results. On the word sets the pre-set full-cache accuracy check failed twice, so every word-set comparison was instead conditioned on full-cache-correct dialogues; this was decided before any eviction method was run on a word set. Accuracy is counted on dialogues the full cache answers: 112/120 (words), 120/120 (digits), 117/120 (words, 1.5B). Methods are compared on the same dialogues with exact McNemar tests (Holm-adjusted within a batch), reported as discordant counts, with Wilson 95% intervals.

3 Results

3.1 Our KVP agents forget

KVP answers 4/112 word facts (3.6%, CI 1.4–8.8) and 5/120 digit facts (4.2%, CI 1.8–9.4) (Table 1). H2O does better on both, 59/112 and 15/120 (56 vs 1 discordant, $p_{\text{Holm}} = 1.6 \times 10^{-15}$; 12 vs 2, $p_{\text{Holm}} = 0.026$): a simple heuristic beats our learned agents here. Whether better-trained agents would

Table 1: Recall (correct/dialogues; $k = 4, 8$; 256 retained tokens per KV head), counted only on dialogues the full cache answers (112/120, 120/120, 117/120), so full cache is 100% by construction. Bold: best usable method per column. Diagnostic rows force part of the fact to stay in the cache (all heads unless stated), which needs its position: a probe, not a method. "Whole user turn" includes chat-template tokens. Restore uses an extra CPU archive. –: not run (no KVP agents at 1.5B).

	Words, 0.5B	Digits, 0.5B	Words, 1.5B
Full cache	112/112 (100%)	120/120 (100%)	117/117 (100%)
Eviction methods (usable in a real chat)			
StreamingLLM	0/112 (0%)	–	0/117 (0%)
H2O	59/112 (53%)	15/120 (12%)	–
KVP	4/112 (4%)	5/120 (4%)	–
StreamingLLM + restore	111/112 (99%)	–	**116/117 (99%)**
KVP + restore	**112/112 (100%)**	**120/120 (100%)**	–
Diagnostic: part of the fact force-kept, using its known position			
StreamingLLM + answer tokens	0/112 (0%)	–	0/117 (0%)
StreamingLLM + answer + linking phrase	105/112 (94%)	–	116/117 (99%)
StreamingLLM + whole user turn	112/112 (100%)	–	116/117 (99%)
KVP + answer tokens	35/112 (31%)	111/120 (92%)	–
KVP + answer + linking phrase	88/112 (79%)	119/120 (99%)	–
KVP + whole user turn, 6 of 48 heads	7/112 (6%)	–	–
KVP + whole user turn, 12 of 48 heads	11/112 (10%)	–	–
KVP + whole user turn, 24 of 48 heads	40/112 (36%)	–	–
KVP + whole user turn, all 48 heads	112/112 (100%)	120/120 (100%)	–

close this gap is open (§2).

3.2 What must survive

Fig. 2 shows one dialogue: the complete fact is in 0, 2 and 48 of the 48 heads under KVP, KVP + answer tokens and KVP + answer + linking phrase, and only the last answers correctly. Over the word set, keeping the answer tokens in all heads gives 35/112 under KVP and 0/112 and 0/117 under StreamingLLM at 0.5B and 1.5B (Table 1). Adding the linking phrase raises this to 88/112, 105/112 and 116/117 (KVP: 55 vs 2 discordant, $p_{\text{Holm}} = 4.6 \times 10^{-14}$; 1.5B: 116 vs 0, $p_{\text{Holm}} = 7.2 \times 10^{-35}$); the whole user turn gives 112/112, 112/112 and 116/117. Under KVP, answer + linking phrase is weaker at the longer gap ($k = 4$: 55/58; $k = 8$: 33/54). The pattern holds at both sizes and supports a role for the linking phrase, but adding it also retains more tokens (3–4 per dialogue): equal-size controls (unrelated tokens, a wrong-subject phrase) are not yet run. On digits, answer-only protection achieves 111/120.

3.3 How many heads

Keeping the whole user turn in 0, 6, 12, 24 and 48 random heads gives 4, 7, 11, 40 and 112 of 112 (Table 1). The same dialogues are used at every level, so we report these counts descriptively. Recall rises steeply between 24 and 48 heads, but we tested no counts in between and used one nested random-subset sequence per dialogue, without repeated subset seeds, so whether a smaller, selected set suffices is open. Head count, layer coverage and the number of forced entries change together.

3.4 Recovery by restoring the retrieved user turn

CPU offloading is known (InfLLM [7], Quest [8]); we use it to test whether restoring the whole turn into all heads, as §3.3 suggests, recovers recall without knowing where the fact is. We copy every computed key and value, with its position, to a CPU archive. When a new user message arrives, a word-overlap retriever picks the best-matching earlier user message, and its tokens are written back whole into every head, displacing the lowest-ranked entries so each head stays at 256 tokens. This brings KVP from 4/112 to 112/112 on words and from 5/120 to 120/120 on digits (Table 1; 108 vs 0 and 115 vs 0 discordant, $p_{\text{Holm}} < 10^{-32}$), and StreamingLLM at 1.5B from 0/117 to 116/117. On the full-cache-correct subsets, restore and whole-turn protection have identical correctness outcomes (zero discordant pairs). Retrieval is easy here: the questions repeat the fact's words, so word overlap finds the right turn in 112/112 and 120/120; this tests recovery after retrieval, not retrieval under paraphrase

(§4). On the full word set, KVP + restore answers 120/120, against 112/120 for the full cache; we have not analysed why. The retained GPU budget is unchanged at each cut, but total memory is not: the CPU archive grows with the conversation. Measured peak GPU memory equals KVP's (about 2.0 GB), and on the word set a dialogue takes about 26% longer (25.8 vs 20.5 s) in our unoptimized code, at about 1,000 tokens.

3.5 Answer-token survival is insufficient

On the first 20 word dialogues (19 answered by the full cache; a descriptive look, not a test), 47% of KVP's head–dialogue pairs (426 of 912) hold at least one answer token when the question arrives, yet recall is 0/19; the answer and the linking phrase are together in only 1 of 912 head–dialogue pairs (Fig. 2 shows one). Answer tokens can survive while recall fails, so their survival alone does not establish recall. Joint survival of answer and linking phrase within a head is a candidate diagnostic we have not yet tested on held-out data.

4 Limitations

- One model family (Qwen2.5, 0.5B and 1.5B); KVP only at 0.5B, with our agents from one training seed. Synthetic dialogues, one fact each; several facts and real conversations are untested. The same dialogue sets were reused across batches, so confirmation on fresh dialogues is still needed.
- We have not compared repeated eviction with one-shot compression of the same transcript, so how much of the failure comes from repetition rather than from the task or our agents is open. Position rescaling and local score pooling did not fix the long-gap failure (earlier bf16 batches, not shown).
- Restore brings back one message (in live chats sometimes an earlier question about the fact), and its CPU archive grows with the conversation. Its word-overlap retriever failed on 4 of 10 paraphrase templates; an embedding retriever found the right message in 1,796 of 1,800, measured as retrieval only, not as final answers.
- At about 1,000 tokens the cache is small next to the weights, so we measure recall under a token budget, not memory savings. The heads in §3.3 were random; whether a subset such as retrieval heads [9] suffices is open.

5 Next steps

Over the next two months we plan to (i) compare repeated eviction with one-shot compression using the same agents, budget and transcripts; (ii) separate the linking phrase from extra retained context with equal-size controls; (iii) test selected and layer-balanced head subsets, with a dependence-aware analysis; (iv) evaluate restore end to end under paraphrase and with several facts, using the embedding retriever; and (v) confirm the main results on fresh dialogues. Restore's memory and latency at scale, and whether agents coordinated across heads keep facts complete without an archive, follow as resources allow. These questions are open; this report describes completed experiments only.

References

[1] L. Moschella, L. Manduchi, O. Sener. Learning to Evict from Key-Value Cache. arXiv:2602.10238, 2026.

[2] Y. Li et al. SCBench: A KV Cache-Centric Analysis of Long-Context Methods. ICLR, 2025.

[3] Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2024.

[4] A. Köpf et al. OpenAssistant Conversations: Democratizing Large Language Model Alignment. NeurIPS Datasets and Benchmarks, 2023.

[5] G. Xiao et al. Efficient Streaming Language Models with Attention Sinks. ICLR, 2024.

[6] Z. Zhang et al. H_2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. NeurIPS, 2023.

[7] C. Xiao et al. InfLLM: Training-Free Long-Context Extrapolation for LLMs with an Efficient Context Memory. arXiv:2402.04617, 2024.

[8] J. Tang et al. Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference. ICML, 2024.

[9] W. Wu et al. Retrieval Head Mechanistically Explains Long-Context Factuality. arXiv:2404.15574, 2024.